

High-Level System Constraints

1. On-Premise or Cloud?

Training is being handled using a hybrid approach (on a local RTX 5070 for test runs of 5–10 epochs) while also leveraging cloud platforms such as Prime Intellect. However, for production inference services, our current and planned approach is to remain **On-Premise (local hardware)** in order to avoid network latency and maintain the privacy of proprietary data within the corporate network.

2. Expected Inference Load

The goal is to achieve a throughput of **more than 1,000 images per second**, ensuring high concurrency.

3. Security Requirements

Since we process Petri dish images and research data, the primary requirement is that **data and models must not leave our controlled infrastructure during inference**. The inference server does not store the analyzed images.

Clarification of the Existing Pipeline

1. How Does DINOv3 Receive Images and Return Predictions?

A request arrives at the server containing a Petri dish image. Our pipeline first uses an **OCR model** to detect written text and identify the type of Petri dish being analyzed. The image is then passed to our DINOv3 model (ViT-Large, 198 million parameters), which operates under a Multi-Task Learning architecture through multiple output heads.

2. Exact Input/Output Format

The input must be a high-resolution image of the complete Petri dish. We are moving away from using multiple crops because they can destroy visual context. While crops are still currently used, we aim to eliminate this practice to optimize inference time.

The output consists of:

- Petri dish classification results
- Precise bounding boxes

The model is currently configured with **100 queries**, but we expect to increase this to **1,000 queries** to detect massive bacterial densities in a single pass.

The attached **JSON** file specifies the exact format in which the inference server response is delivered to the end-user system.

3. Which Heads Are Used and What Are the Classes?

The model uses 8 simultaneous heads for classification and detection tasks, configured with `num_classes`.

These heads correspond directly to evaluations on our internal datasets: **YM, EC, TC, XSA, LM, ETB, BC, LS**.

Annotation Tool Preference

1. Labelbox, Label Studio, or Custom UI?

We prefer moving toward **Label Studio (open-source)** hosted on our own servers. This avoids excessive commercial licensing costs and allows easy integration through APIs with our training pipelines.

2. Who Performs Annotation?

Annotations are performed by an internal team of **laboratory analysts**. The core strategy is not to annotate the entire volume of images, but rather to use an intelligent filtering process.

Decision on Advanced Features

1. Ensemble, Hybrid Sampling, or Pseudo-Labeling?

We want to implement **uncertainty-based Hybrid Sampling (Active Learning)** so that only images with low model confidence are sent for human review. This is expected to reduce manual annotation work by 60%–80%.

Additionally, we are interested in using **Self-Supervised Learning (SSL)** or **Pseudo-Labeling** to leverage our historical archive of thousands of unlabeled images.

2. Ensemble Strategy and Latency

We are not seeking a traditional large-scale ensemble (3–5 models running simultaneously) because this negatively impacts RAM usage and latency.

Instead, we prefer a **Mixture of Experts (MoE)** strategy integrated with the OCR system. This allows us to have specialized expert models without significantly increasing memory consumption or response time, since only the portion of the network relevant to a specific Petri dish is activated during inference.